CSA1737 – ARTIFICIAL INTELLIGENCE

LIST OF EXPERIMENTS

16. Write the Python program to implement BFS

AIM:

To write a Python program to implement Breadth-First Search (BFS) for traversing a graph.

ALGORITHM:

1. Start the program.
2. Create a graph using an adjacency list.
3. Initialize a queue with the starting node.
4. Mark the starting node as visited.
5. Remove a node from the queue and display it.
6. Add all unvisited adjacent nodes to the queue.
7. Repeat until the queue becomes empty.
8. Stop the program.

CODE:

```
from collections import deque
graph={'A':['B','C'],'B':['D','E'],'C':['F'],'D':[],'E':[],'F':[]}
def bfs(start):
    visited={start}
    queue=deque([start])
    while queue:
        node=queue.popleft()
        print(node,end=" ")
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
bfs('A')
```

OUTPUT:

```
A B C D E F
```

17. Write the Python program to implement DFS

AIM:

To write a Python program to implement Depth-First Search (DFS) for traversing a graph.

ALGORITHM:

1. Start the program.
2. Create a graph using an adjacency list.
3. Initialize an empty set to store visited nodes.
4. Visit the starting node and display it.
5. Mark the node as visited.
6. Recursively visit each unvisited adjacent node.
7. Repeat until all reachable nodes are visited.
8. Stop the program.

CODE:

```
graph={'A':['B','C'],'B':['D','E'],'C':['F'],'D':[],'E':[],'F':[]}
visited=set()
def dfs(node):
    if node not in visited:
        print(node,end=" ")
        visited.add(node)
        for neighbor in graph[node]:
            dfs(neighbor)
dfs('A')
```

OUTPUT:

```
A B D E C F
```

18. Write the Python program to implement Travelling Salesman Problem

AIM:

To write a Python program to solve the Travelling Salesman Problem (TSP) by finding the minimum-cost tour that visits every city exactly once and returns to the starting city.

ALGORITHM:

1. Start the program.
2. Define the cost matrix representing distances between cities.
3. Fix the starting city.
4. Generate all possible orders of visiting the remaining cities.
5. Calculate the total cost of each tour.
6. Find the tour with the minimum cost.
7. Display the optimal tour and its cost.
8. Stop the program.

CODE:

```
from itertools import permutations
cost=[[0,10,15,20],[10,0,35,25],[15,35,0,30],[20,25,30,0]]
n=len(cost)
min_cost=float('inf')
best_path=[]
for path in permutations(range(1,n)):
    tour=(0,)+path+(0,)
    total=sum(cost[tour[i]][tour[i+1]] for i in range(len(tour)-1))
    if total<min_cost:
        min_cost=total
        best_path=tour
print("Best Tour:",best_path)
print("Minimum Cost:",min_cost)
```

OUTPUT:

```
Best Tour: (0, 1, 3, 2, 0)
Minimum Cost: 80
```

19. Write the Python program to implement A* algorithm

AIM:

To write a Python program to implement the A* search algorithm to find the shortest path from a start node to a goal node using path cost and heuristic values.

ALGORITHM:

1. Start the program.
2. Define the graph with edge costs.
3. Define the heuristic value for each node.
4. Initialize the open list with the starting node.
5. Select the node with the lowest \(f(n)=g(n)+h(n)\).
6. If the goal is reached, display the path and total cost.
7. Otherwise, calculate the costs of neighboring nodes and update the open list.
8. Repeat until the goal is reached or no path exists.
9. Stop the program.

CODE:

```
import heapq
graph={'A':[('B',1),('C',4)],'B':[('D',2),('E',5)],'C':[('E',1)],'D':[('G',5)],'E':[('G',2)],'G':[]}
h={'A':6,'B':5,'C':3,'D':4,'E':2,'G':0}
def astar(start,goal):
    queue=[(h[start],0,start,[start])]
    visited=set()
    while queue:
        f,g,node,path=heapq.heappop(queue)
        if node==goal:
            return path,g
        if node in visited:
            continue
        visited.add(node)
        for neighbor,cost in graph[node]:
            if neighbor not in visited:
                new_g=g+cost
                heapq.heappush(queue,(new_g+h[neighbor],new_g,neighbor,path+[neighbor]))
    return None,None
path,cost=astar('A','G')
if path:
    print("Shortest Path:",path)
    print("Total Cost:",cost)
else:
    print("No path exists")
```

OUTPUT:

```
Shortest Path: ['A', 'C', 'E', 'G']
Total Cost: 7
```

20. Write the Python program for Map Coloring to implement CSP

AIM:

To write a Python program to solve the Map Coloring problem using a Constraint Satisfaction Problem (CSP), ensuring that no two adjacent regions have the same color.

ALGORITHM:

1. Start the program.
2. Define the regions and available colors.
3. Define the neighboring regions.
4. Select an uncolored region.
5. Assign a color that does not conflict with its neighbors.
6. If no valid color is available, backtrack.
7. Repeat until all regions are colored.
8. Display the solution.
9. Stop the program.

CODE:

```
regions=['A','B','C','D']
colors=['Red','Green','Blue']
neighbors={'A':['B','C'],'B':['A','C','D'],'C':['A','B','D'],'D':['B','C']}
assignment={}
def is_safe(region,color):
    return all(assignment.get(neighbor)!=color for neighbor in neighbors[region])
def solve(index):
    if index==len(regions):
        return True
    region=regions[index]
    for color in colors:
        if is_safe(region,color):
            assignment[region]=color
            if solve(index+1):
                return True
            del assignment[region]
    return False
if solve(0):
    for region in regions:
        print(region,":",assignment[region])
else:
    print("No solution exists")
```

OUTPUT:

```
A : Red
B : Green
C : Blue
D : Red
```